```
import cv2
import numpy as np
import matplotlib.pyplotas plt
from google.colab import files
```

```
uploaded = files.upload()

video_name = list(uploaded.keys())[0]
print("Uploaded video:", video_name)
```

```
Choose Files 13473843_…_25fps.mp4
13473843_3840_2160_25fps.mp4(video/mp4) - 56342311 bytes, last modified: 9/12/2026
100% done
Saving 13473843_3840_2160_25fps.mp4 to 13473843_3840_2160_25fps.mp4
```

```
cap = cv2.VideoCapture(video_name)

if not cap.isOpened():
    print("Error: Could not open the video.")
else:
    print("Video opened successfully.")

fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("FPS:", fps)
print("Width:", width)
print("Height:", height)
```

```
Video opened successfully.
FPS: 25.0
Width: 3840
Height: 2160
```

```
ret, frame1 = cap.read()
ret2, frame2 = cap.read()

if not ret or not ret2:
    print("Error: Could not read video frames.")
else:
    print("Two consecutive frames loaded successfully.")

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
plt.title("Frame 1")
```

```
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB))
plt.title("Frame 2")
plt.axis("off")

plt.show()
```

```
Two consecutive frames loaded successfully.
```




```
# Parameters for Shi-Tomasi corner detection
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

# Parameters for Lucas-Kanade Optical Flow
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT ,
              10, 0.03)
)

# Detect feature points in first frame
p0 = cv2.goodFeaturesToTrack(gray1, mask=None, **feature_params)

if p0 is None:
    print("No feature points detected.")
else:
    print("Number of feature points:", len(p0))

# Calculate optical flow
p1, st, err = cv2.calcOpticalFlowPyrLK(
    gray1,
```

```
    p0,
    None,
    **lk_params
)

# Select successfully tracked points
if p1 is not None:
    good_new = p1[st == 1]
    good_old = p0[st == 1]

    print("Successfully tracked points:", len(good_new))
```

```
Number of feature points: 100
Successfully tracked points: 100
```

```
# Create a copy of the second frame
lk_result = frame2.copy()

# Draw motion vectors
for new, old in zip(good_new, good_old):
    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    cv2.arrowedLine(
        lk_result,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2,
        tipLength=0.3
    )

    cv2.circle(
        lk_result,
        (int(x_new), int(y_new)),
        4,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(lk_result, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Optical Flow")
plt.axis("off")
plt.show()
```

Lucas-Kanade Sparse Optical Flow



```
flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

print("Farneback Dense Optical Flow calculated successfu lly.")
```

```
Farneback Dense Optical Flow calculated successfully.
```

```
# Convert flow vectors from Cartesian coordinates  to polar coordinates
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# Create HSV image
hsv = np.zeros_like(frame1)
```

Ques 1. What is Optical Flow? How is it used in computer vision?

Ans. Optical Flow is a computer vision technique used to estimate the apparent motion of objects, surfaces, or the camera between two consecutive video frames. It analyzes changes in image pixels to determine the direction and amount of motion. Optical flow is used in applications such as object tracking, autonomous driving, video surveillance, robotics, gesture recognition, sports analytics, and augmented reality.

Ques 2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

Ans. Lucas-Kanade is a Sparse Optical Flow algorithm that estimates the motion of selected feature points between consecutive frames. First, important feature points such as corners are detected in the first frame. The algorithm then assumes that the pixel intensity of a point remains approximately constant while moving and that nearby pixels have similar motion. Using these assumptions, it calculates the displacement of the selected points in the next frame and represents their movement using motion vectors or trajectories.

Ques 3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

Ans. Dense Optical Flow estimates the motion for almost every pixel in an image or video frame. In contrast, Sparse Optical Flow calculates motion only for selected feature points. Dense optical flow provides a complete motion field and is useful for detailed motion analysis, while sparse optical flow requires less computation and is useful for tracking important points or objects.

Ques 4. Compare the Lucas-Kanade and Farneback optical flow algorithms.

Ans. Lucas-Kanade is a Sparse Optical Flow algorithm that tracks selected feature points, making it computationally efficient and suitable for object or feature tracking. Farneback is a Dense Optical Flow algorithm that estimates motion across the image and therefore provides more complete motion information. Lucas-Kanade generally has lower computational complexity, whereas Farneback provides more detailed information about motion throughout the scene.

Ques 5. What assumptions are made while computing optical flow?

Ans. Optical flow computation is based on several important assumptions. The first is the brightness or intensity constancy assumption, which means that the appearance of a moving point remains approximately the same between consecutive frames. The second is the assumption of small motion between frames. Another assumption is that neighboring pixels generally have similar motion. These assumptions allow the algorithm to estimate the movement of image points.